

1.5em 0pt

1 Introduction

1.1 Problem statement

The main goal is to estimate the arboricity in a graph G $\lambda(G)$ - a metric that describes the minimal number of edge-disjoint forests that G can be split into. Existing algorithms achieved results listed in the table 1:

Table 1: Comparison of existing algorithms for arboricity and degeneracy estimation.

Algorithm / Authors	Target Metric	Approx. Ratio	Query Complexity	Time / Round Complexity	Space Complexity
Gabow (1998) [6]	Arboricity	1	–	$\tilde{O}(m^{3/2})$	$\text{Poly}(n)$
Matula & Beck (1983) [10]	Degeneracy & Arboricity	1 (for Degeneracy), 2 (for Arboricity)	–	$O(n + m)$	$O(n + m)$
King, Thomo, Yong (2022) [8]	Degeneracy	$1 + \epsilon$	$\tilde{O}(n)$	$\tilde{O}(n)$	$\tilde{O}(n)$
Bhattacharya et al. (2015) [3]	Densest Subgraph / Arboricity	$4 + \epsilon$ (integer arboricity) $2 + \epsilon$ (densest subgraph)	$\tilde{O}(n)$	$\tilde{O}(n)$ Static, $\tilde{O}(1)$ Dynamic amortized	$\tilde{O}(n)$
Eden, Mossel, Ron (2022) [5]	Arboricity	$O(\log^2 n)$	$\tilde{O}(\frac{n}{\lambda})$	$\tilde{O}(\frac{n}{\lambda})$	$\tilde{O}(\frac{n}{\lambda})$
Dai, Ghaffari, Portmann (2025) [4]	Arboricity	$O(1)$ (120 in the algorithm)	$\tilde{O}(\frac{n}{\lambda})$	$\tilde{O}(\frac{n}{\lambda})$	$\tilde{O}(\frac{n}{\lambda})$
Proposed algorithm	Arboricity	$(1 + \epsilon) \pm 1$	$\tilde{O}(n)$	$\tilde{O}(n)$	$\tilde{O}(n)$

1.2 Graph input model

In the proposed algorithm the graph is given in form of a incidence or adjacency-list model, where it takes $O(1)$ time to access the i -th neighbour of the j -th vertex, and get the vertex degree by counting the number of neighbours.

1.3 Graph metrics

Main graph metrics that we will consider later are following:

$$\rho(G) = \max_{S \subseteq V(G)} \frac{|E(S)|}{|S|} - \text{fractional density};$$

$$\gamma(G) = \max_{S \subseteq V(G), |S| \geq 2} \frac{|E(S)|}{|S| - 1} - \text{fractional arboricity};$$

$$\lambda(G) = \max_{S \subseteq V(G), |S| \geq 2} \lceil \frac{|E(S)|}{|S| - 1} \rceil - \text{integer arboricity};$$

,

We can derive relations between them:

$$\lambda(G) = \lceil \gamma(G) \rceil$$

$$\gamma(G) - \rho(G) \leq \frac{|E(S)|}{|S| - 1} - \frac{|E(S)|}{|S|} = \frac{E(S)}{|S|(|S| - 1)} \leq \frac{1}{2}$$

because $|E(S)| \leq \frac{|S|(|S| - 1)}{2}$ (maximum number of edges when S is a clique)

Therefore by estimating the fractional density with factor $(1 + \varepsilon)$ we can approximate integer arboricity like $\lambda(G) = \lceil \gamma(G) \rceil \leq \lceil \rho(G) + \frac{1}{2} \rceil \leq \lambda(G)(1 + \varepsilon) + 1$.

2 Algorithm

2.1 Decision problem formulation

Similarly to Dai et al.[4] we formulate arboricity approximation scheme as the decision problem by using a comparator $Test(G, \rho, \varepsilon)$ that takes graph G , approximation constant $\varepsilon \in (0; 0.5)$, and current approximation of the fractional density ρ^* as an input. We iterate over $\rho_i := \frac{n}{(1+\varepsilon)^i}$ from $i = 0$ while $\rho_i > \frac{1}{n}$. On the i -th iteration we run $Test(G, \rho_i, \varepsilon)$, it constructed in such way that it returns YES if $\rho(G) \leq \rho_i$ and NO if $\rho(G) \geq (1 + \varepsilon)\rho_i$.

We continue taking iterations until first NO verdict and return approximation as the $\lceil \rho_i + \frac{1}{2} \rceil$ on the last ρ_i with YES.

2.2 Test() details

Algorithm of the $Test(G, \rho_i, \varepsilon)$ can be split into following 3 simple steps:

1. **High density check:** If number of edges in a graph is already greater than $\rho_i |V(G)|$, then we can return NO straight away and go to the next iteration;
2. **Construction of the sparse graph H based on G :** We get sparse graph H with expected $O(n \log n)$ number of edges by sampling them from graph G ;
3. **Fractional density estimation in H :** We use Multiplicative weights update(MWU) scheme, initially proposed by [?], to solve fractional edge orientation LP and in the end get $(1 + \varepsilon)$ approximation of the maximal outdegree in a graph with minimal orientation. If $D' \leq (1 + \varepsilon_1)((1 + \varepsilon_2)\lambda p)$, ($(1 + \varepsilon_2)\lambda p$ is threshold maximum outdegree in H) return YES, else NO;

High level depiction of the algorithm can be seen on the Figure 1. Before describing the following steps we also need to fix some constants, which values will be justified in the following steps of the described algorithm. Among them are $\varepsilon_1 = \frac{\varepsilon}{4}, \varepsilon = \frac{\varepsilon}{4}, c = \frac{48}{\varepsilon^2}, p_i = \min(1, \frac{c \log n}{\rho_i}), \rho = (1 + \varepsilon_2)c \log n$ (or $\rho = \rho_i$ if $p_i = 1$), $T = \lceil \frac{16 \log n}{\varepsilon} \rceil$.

2.2.1 High density check

In $O(n)$ time we can get the sum of all of the vertex degrees in a graph that is equal to the doubled number of edges in it $2m$. If found $m \geq \rho_i n$, then by result of C. St. J. A. Nash-Williams [?] there must be a ρ -dense core that bring us to the immediate NO answer. Otherwise we continue inspection of the graph G by taking steps below.

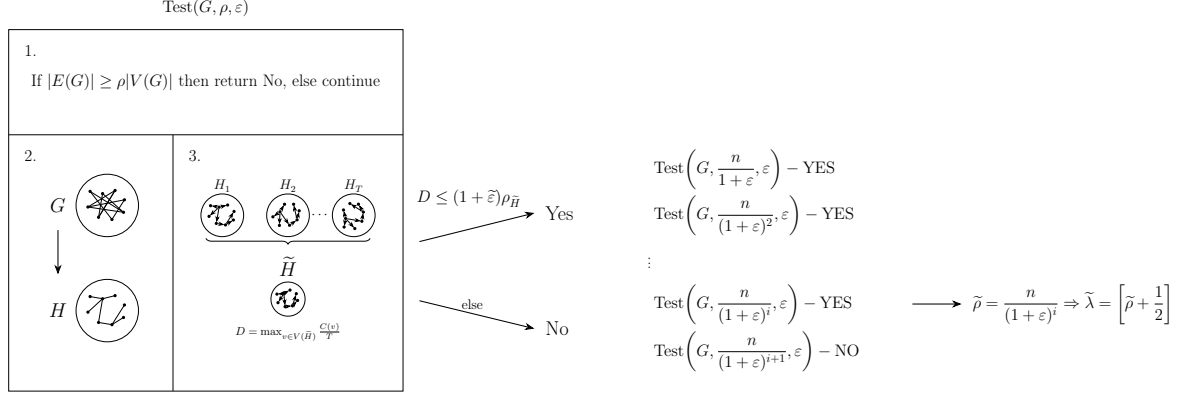


Figure 1: High level scheme of the proposed algorithm

2.2.2 Construction of the sparse graph H based on G

The main goal of this step is to acquire a sparse approximation H of the original graph G in $\tilde{O}$ time and query complexity. This step is based on the previous findings of Batagelj et al.[2] where they significantly improved the time needed to generate new graph from existing one by using steps generated from Geometrical distribution instead of probing all of the edges in $O(m)$ time. We use very similar scheme that will be explained below.

Firstly, we initialise an graph H with n vertices and no edges. Then if p -probability that invertly depends on the ρ equal to one, then we just add all of the edges from G , without duplication. Otherwise, for every $v \in V(G)$ we initialise $i = 0$ and while $i < \deg(v)$ then we generate a step from geometrical distribution $\delta \sim \text{Geom}(p)$ and increase i by δ . If updated delta is lower then the number of neighbours of v , then we add edge $\{v, w\}$ where w is the i -th neighbour of v if the id of w is greater then id of v to avoid duplicates. After the procedure is done we are left with the graph H that has an expected number of edges $O(n \log n)$, to keep the complexity of the whole algorithm no greater than $O(n)$ we also stop when we hit the $n \log n$ number of vertices and return No if the algorithm have not reached its end by this point. For better understanding H creation step is also provided in a form of algorithm 1.

Algorithm 1 Construction of sparse H from G

Require: Graph G in Adjacency-List Query model, density threshold ρ_i , precision parameter ϵ

Ensure: Sparse sampled graph H or halt with NO verdict

```

1:  $E_H \leftarrow \emptyset$ 
2:  $p \leftarrow \min\left(1, \frac{c \ln n}{\rho}\right)$ 
3: if  $p \geq 1$  then
4:   return  $H = (V(G), E(G))$ 
5: end if
6:  $M_{\max} \leftarrow 2cn \ln n$ 
7: for each vertex  $v \in V(G)$  do
8:    $i \leftarrow 0$ 
9:   while true do
10:    Sample  $\delta \sim \text{Geom}(p)$ 
11:     $i \leftarrow i + \delta$ 
12:    if  $i > \deg(v)$  then
13:      break
14:    end if
15:     $w \leftarrow \text{Neighbor}(v, i)$ 
16:    if  $\text{id}(w) > \text{id}(v)$  then
17:       $E_H \leftarrow E_H \cup \{\{v, w\}\}$ 
18:      if  $|E_H| > M_{\max}$  then
19:        return No
20:      end if
21:    end if
22:  end while
23: end for
24: return  $H = (G(V), E_H)$ 

```

2.2.3 Fractional density estimation in H

An important result by S.L. Hakimi [7] is used for this density estimation step: G has a maximum subgraph density D if and only if you can orient the edges of G such that no vertex has an out-degree strictly greater than D .

Because of this equivalence we can use the Multiplicative Weights Update (MWU) scheme, initially proposed by Plotkin et al. [?]. In the original paper this approach was used for fractional packing and provided $(1 + \epsilon)$ -approximation of the x in a task of solving LP $Ax \leq b, y^T Ax \rightarrow \min$ much faster than the other methods. This procedure allows us to fractionally orient the edges so the out degree could not be greater than the stated ρ , for that we need n constraints for each of the vertices. On each iteration of the MWU scheme we assign to each edge $e = \{u, v\}$ a simplex $x_{eu} + x_{ev} = 1$. For each vertex v we also assign weight $W(v)$ and try to solve following optimisation task: $W(u)x_{eu} + W(v)x_{ev} \rightarrow \min$. From properties of simplex we know that the optimal solution is either $x_{eu} = 1, x_{ev} = 0$ or $x_{eu} = 0, x_{ev} = 1$, so

solution is orientation of the edge to the vertex with the smaller weight. Simillar observation was made by Bahmani et al. [1]. In order to achieve that, on each iteration we need to count the load of each vertex $L(v)$, that is the number of edges pointed to it currently, and keep $C(v)$ - cumulative load to get the mean value in the end. $W(v)$ is updated like following $W(v) := W(v)(1 + \gamma)^{L(v)}$ to provide exponential load penalty to vertices that had a lot of edges oriented to then on this step. In the end, following [9], the $(1 + \varepsilon)$ approximation of D is obtained as $\tilde{D} = \max_{v \in V(H)} \frac{C(v)}{T}$, where T is the number of iterations. In the beginning for all of the vertices $W(v) = 1, C(v) = 0$ and at the start of each iteration $L(v)$.

This step gives $(1 + \varepsilon)$ approximation of D . Now we can use it to get the verdict by comparing $\tilde{D}$ to equivalent of ρ threshold, nut in our sparse graph H - the $\widetilde{rho}$. If $\tilde{D} \leq (1 + \varepsilon_2)\tilde{\rho}$ then we return Yes, else - No. All of the above can be depicted by the following algorithm 2

Algorithm 2 MWUDensityTest($H, \tilde{\rho}, \varepsilon$)

Require: Sparse graph H , fractional density threshold $\tilde{\rho}$, approximation parameter ε

Ensure: **Yes** if maximum subgraph density of H is $\leq (1 + \varepsilon_2)\tilde{\rho}$, else **No**

```

1:  $T \leftarrow \lceil \frac{c \ln |V|}{\varepsilon^2} \rceil$ 
2:  $\varepsilon_2 \leftarrow \frac{\varepsilon}{4}$ 
3: for each  $v \in V$  do
4:    $W(v) \leftarrow 1$ 
5:    $C(v) \leftarrow 0$ 
6: end for
7: for  $t = 1$  to  $T$  do
8:   for each  $v \in V$  do
9:      $L(v) \leftarrow 0$ 
10:  end for
11:  for each edge  $e = \{u, v\} \in E_H$  do
12:    if  $W(u) \leq W(v)$  then
13:       $L(u) \leftarrow L(u) + 1$ 
14:    else
15:       $L(v) \leftarrow L(v) + 1$ 
16:    end if
17:  end for
18:  for each  $v \in V$  do
19:     $C(v) \leftarrow C(v) + L(v)$ 
20:     $W(v) \leftarrow (1 + \varepsilon_2)^{C(v)}$ 
21:  end for
22: end for
23:  $\tilde{D} \leftarrow \max_{v \in V} \frac{C(v)}{T}$ 
24: if  $\tilde{D} \leq (1 + \varepsilon_2)\tilde{\rho}$  then
25:   return Yes
26: else
27:   return No
28: end if

```

3 Correctness and error bound estimation

Error estimation will be soon added.

4 Asymptotic analysis

4.1 Time complexity

By utilising the stepping scheme of the algorithm we take a number of iterations to get the first NO verdict, we can estimate the number of iterations taken as $\log_{(1+\varepsilon)} n = O(\frac{\log n}{\varepsilon})$ that becomes a multiplicative factor that contributes to the overall time complexity.

Step 1 of the Test() takes $O(n)$ queries that run in $O(1)$ time from the model of input graph that we use. These queries are used to get degrees of all of the vertices and get the doubled number of edges in G . Step 2 takes up $O(n \log n)$ queries judging from the resulting expected number of edges in a graph, where each requires $O(1)$ time complexity query.

Therefore resulting time and query complexity is $O(n \text{ poly}(\log n)) = \tilde{O}(n)$

4.2 Additional space

Additional space is used to store the sparse graph H with $O(n \log n)$ expected edges and n nodes that results in $\tilde{O}(n)$ additional space used.

References

- [1] Bahman Bahmani, Ashish Goel, and Kamesh Munagala. Efficient primal-dual graph algorithms for mapreduce. In *Algorithms and Models for the Web Graph: 11th International Workshop, WAW 2014, Beijing, China, December 17-18, 2014, Proceedings*, page 59–78, Berlin, Heidelberg, 2014. Springer-Verlag.
- [2] Vladimir Batagelj and Ulrik Brandes. Efficient generation of large random networks. *Phys. Rev. E*, 71:036113, Mar 2005.
- [3] Sayan Bhattacharya, Monika Henzinger, Danupon Nanongkai, and Charalampos E. Tsourakakis. Space- and time-efficient algorithm for maintaining dense subgraphs on one-pass dynamic streams. In *Proceedings of the 47th Annual ACM Symposium on Theory of Computing (STOC)*, pages 173–182, 2015.
- [4] Jiangqi Dai, Mohsen Ghaffari, and Julian Portmann. Constant approximation of arboricity in near-optimal sublinear time. *arXiv preprint arXiv:2512.18416*, 2025.
- [5] Talya Eden, Saleet Mossel, and Dana Ron. Approximating the arboricity in sublinear time. In *Proceedings of the 33rd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2404–2425, 2022.
- [6] Harold N. Gabow. Algorithms for graphic polymatroids and parametric-sets. *Journal of Algorithms*, 26(1):48–86, 1998.

- [7] S.L. Hakimi. On the degrees of the vertices of a directed graph. *Journal of the Franklin Institute*, 279(4):290–308, 1965.
- [8] Valerie King, Alex Thomo, and Quinton Yong. Computing $(1 + \epsilon)$ -approximate degeneracy in sublinear time. *arXiv preprint arXiv:2211.04627*, 2022.
- [9] Jan Karel Lenstra, David B. Shmoys, and Eva Tardos. Approximation algorithms for scheduling unrelated parallel machines. In *Proceedings of the 28th Annual Symposium on Foundations of Computer Science*, SFCS '87, page 217–224, USA, 1987. IEEE Computer Society.
- [10] David W. Matula and Leland L. Beck. Smallest-last ordering and clustering and graph coloring algorithms. *Journal of the ACM (JACM)*, 30(3):417–427, 1983.